

PRACTICAL_3

3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
GNU nano 8.7.1 Practical3.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(10);
    }
    else
    {
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());

        wait(NULL);
    }

    return 0;
}
```

```
ketana_sindhu@KetaZone:~$ nano Practical3.c
ketana_sindhu@KetaZone:~$ gcc Practical3.c -o p3
ketana_sindhu@KetaZone:~$ ./p3
Parent Process
PID : 1615
Child Process
PID : 1616
PPID : 1615
ketana_sindhu@KetaZone:~$ |
```

Practical_4

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```
GNU nano 8.7.1 Practical4.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;

    for(i = 0; i < 3; i++)
    {
        if(fork() == 0)
        {
            printf("Child %d PID=%d\n", i + 1, getpid());
            return 0;
        }
    }

    for(i = 0; i < 3; i++)
    {
        wait(NULL);
    }

    printf("All children completed.\n");

    return 0;
}
```

```
ketana_sindhu@KetaZone:~$ nano Practical4.c
ketana_sindhu@KetaZone:~$ gcc Practical4.c -o p4
ketana_sindhu@KetaZone:~$ ./p4
Child 1 PID=1689
Child 2 PID=1690
Child 3 PID=1691
All children completed.
ketana_sindhu@KetaZone:~$ |
```